Style Radar

SQUAD GUATOQUE JIRAGUAS (GJ)
DOSW · SPRINT 02 · 2026

SQUAD GUATOQUE JIRAGUAS · ESPECIFICACIÓN TÉCNICA · SPRINT 02

DOCUMENTO TÉCNICO

Contexto, arquitectura hexagonal, endpoints, eventos publicados y consumidos, modelo de datos y decisiones técnicas del microservicio **GJ-Chat-Service** (Puerto 8320 · Paquete 3) al cierre del Sprint 2, dentro del ecosistema distribuido de StyleRadar.

MS-CHAT :8320 | PAQUETE 3 OFICIAL | ARQUITECTURA HEXAGONAL | POSTGRESQL 15 (CHAT_DB)

SPRING BOOT 3.2.12 | JAVA 17 | RABBITMQ 3.13 + DLQ | JWT HS256 | JACOCO 96 % INSTRUCCIONES

FICHA TÉCNICA · ARQUITECTURA DE SOFTWARE

PROYECTO / ASIGNATURA	MICROSERVICIO ASIGNADO	LÍDER DE SQUAD	RESPONSABLES
StyleRadar · DOSW 2026	GJ-Chat-Service (Puerto 8320 · Paquete 3)	Paula Diaz	Jose Martinez (Líder Backend) · Santiago García (Desarrollo)

SR · DOSW · 2026

00 ESTRUCTURA DEL DOCUMENTO

Este documento reúne la especificación técnica del microservicio **GJ-Chat-Service** (`ms-chat`) a cierre del Sprint 2 y responde a la tarea GJ-100 del tablero del Squad Guatoque Jiraguas. Es la continuación de `SR_GJ_DocumentoTecnico_v1` (GJ-10) y sigue la misma estructura que el documento de ms-notificaciones, para que los dos servicios del squad tengan el mismo nivel de detalle.

TABLA DE CONTENIDO OFICIAL

SECCIÓN	TÍTULO Y ALCANCE	DESCRIPCIÓN TÉCNICA DE LOS CONTENIDOS
01	**Contexto y drivers**	Problema, qué hace y qué no hace (C2C, B2C), atributos de calidad (ISO/IEC 25010), restricciones y estado de implementación por sprint.
02	**Arquitectura hexagonal**	Capas, diagrama de componentes, flujo de enviar un mensaje, estructura de paquetes y deuda arquitectónica.
03	**Endpoints y Swagger**	Contrato REST, seguridad JWT, códigos de error, ejemplo de petición y evidencia de Swagger.
04	**Eventos**	`MensajeEnviado` publicado y `PrendaSegundaManoVendida` consumido: exchange, colas, routing keys, cola de errores e idempotencia.
05	**Modelo de datos**	Tablas `conversaciones`, `mensajes` y `processed_events` en PostgreSQL, restricciones e índices.
06	**Decisiones técnicas**	Metodología multicriterio y registros ADR-001 a ADR-004 con alternativas descartadas.
07	**Riesgos y mitigación**	Riesgos reales, estado de la mitigación y controles de calidad vigentes.
08	**Conclusiones y equipo**	Stack con versiones fijas, verificación realizada, pendientes y responsables.

Convención de lectura: salvo que se indique lo contrario, este documento describe el estado de la rama `feature/sprint-2` de GJ-Chat-Service al 7 de noviembre de 2026, pendiente de pull request a `develop`. Las capacidades planificadas para el Sprint 3 se marcan como tales.

CAMBIOS DEL SPRINT 2 RESPECTO A LA V1

TEMA	V1 (SPRINT 1)	V2 (SPRINT 2)
Puerto y broker (GJ-94)	Puerto 8320 definido; sin broker.	Puerto `8320`; RabbitMQ 3.13 en `docker-compose.yml`; variables `RABBITMQ_*`; el servicio espera al broker sano.
Eventos	Solo contratos propuestos.	Publica `chat.mensaje.enviado`; consume `segunda-mano.prenda.vendida`; idempotencia con `processed_events` (migración V2) y cola de errores.
Casos de uso	4	5 (se añade `CloseConversationsOnSale`).
Código	Nombres de clases en español.	Código en inglés; el contrato externo (rutas, JSON, tablas, valores) no cambia.
Pruebas	74 pruebas.	85 pruebas sin fallos; JaCoCo 96 % de instrucciones; colección Postman de 13 pruebas.

01 CONTEXTO Y DRIVERS DEL MICROSERVICIO

StyleRadar es una plataforma de moda local para Bogotá con inventario multi-sede. Este servicio permite que quien quiere comprar una prenda de segunda mano converse con su dueño para acordar la entrega, sin salir de la plataforma.

1.1 PLANTEAMIENTO DEL PROBLEMA

Un comprador que ve una prenda de segunda mano necesita preguntarle al vendedor por el estado, el lugar y el momento de la entrega. Al pasar de monolito a microservicios, la publicación y los usuarios pertenecen a otros servicios y consultar sus tablas violaría **Database-per-Service**. Por eso `ms-chat` guarda solo los ids, pide la publicación por REST cuando necesita decidir en el momento y reacciona por eventos cuando la prenda se vende.

QUÉ HACE ESTE MICROSERVICIO	QUÉ NO HACE (DELEGADO)
Inicia una conversación por publicación y comprador; si ya existe, devuelve la misma.	No crea ni modifica publicaciones: lo hace el servicio de segunda mano.
Guarda y entrega los mensajes de cada conversación solo a sus dos participantes.	No autentica ni emite tokens: valida el JWT que emite el servicio de identidad.
Valida contra el servicio de segunda mano que la publicación exista, esté disponible y no sea del comprador.	No envía notificaciones: publica `MensajeEnviado` y lo atiende `ms-notificaciones`.
Publica `chat.mensaje.enviado` y cierra salas al recibir `segunda-mano.prenda.vendida`.	No gestiona pagos ni la entrega física de la prenda.
Chat C2C comprador–vendedor (implementado).	Chat B2C con almacenes (GJ-56), donaciones (GJ-57) y tiempo real por WebSocket (GJ-54) son del Sprint 3: no implementados.

1.2 ATRIBUTOS DE CALIDAD CLAVE (ISO/IEC 25010)

ATRIBUTO	ESCENARIO OBJETIVO	IMPACTO EN EL DISEÑO
Fiabilidad	Un evento de venta repetido no cierra salas dos veces ni deja estado inconsistente.	Tabla `processed_events` (`event_id` como llave primaria) y verificación previa en el caso de uso.
Tolerancia a fallos	Un mensaje inválido no bloquea la cola y una caída del broker no rompe el envío de mensajes.	`styleradar.dlx` y `q.chat.segunda-mano.dlq`; el publicador registra el error y sigue; Feign con tiempos límite y `503` claro.
Seguridad	Cada usuario solo ve y escribe en sus conversaciones.	JWT HS256 stateless; el usuario sale del token, nunca de la URL; una conversación ajena responde 404.
Mantenibilidad	Las reglas se prueban sin levantar Spring.	Dominio puro, puertos de interfaz con `*UseCaseImpl`; JaCoCo mínimo de 85 %.
Interoperabilidad	Otros servicios entienden los eventos y los errores.	Sobre común de evento (`eventId`, `eventType`, `version`…) y errores RFC 7807.
Portabilidad	El servicio se levanta con un solo comando.	Dockerfile multi-stage con usuario no root y `docker compose up` con PostgreSQL y RabbitMQ sanos.

1.3 RESTRICCIONES TÉCNICAS DEL PROYECTO

- Arquitectura hexagonal votada por el curso: puertos de entrada como interfaz con su `Impl` registrada en `BeanConfig` (sin `@Service`) y dominio sin frameworks.
- Puerto oficial `8320`, base propia `chat_db` (PostgreSQL 15) y broker compartido RabbitMQ.
- Código en inglés; el contrato externo (rutas, nombres de campos JSON, tablas y valores) se conserva en español.
- Secretos fuera del repositorio; solo se versiona `.env.example`.
- Cada diagrama acompañado de explicación y justificación; cada tarea terminada con evidencia.

1.4 ESTADO DE IMPLEMENTACIÓN FRENTE AL PLAN

El tablero reserva la funcionalidad por sprint. Una tarea se da por entregada cuando tiene su evidencia en Jira; la columna de estado distingue lo entregado de lo que sigue abierto.

SPRINT	TAREAS GJ	ALCANCE PLANIFICADO	ESTADO EN EL REPOSITORIO
Sprint 1	GJ-3 a GJ-12	Repositorio, scaffolding hexagonal, Docker, Swagger, README, diagramas, documento técnico, análisis de tecnologías y acuerdo de eventos.	**Entregado.** Todas en Listo.
Sprint 2	GJ-14, GJ-15, GJ-16	Migrar dominio, definir puertos y casos de uso, persistencia con JPA.	**Entregado** con evidencia.
Sprint 2	GJ-17, GJ-18, GJ-19	Validar la publicación, migrar endpoints, seguridad JWT.	**Implementado y verificado** (401, 403, 404, 422; endpoints; Postman). Falta adjuntar Swagger y pipeline de GJ-17.
Sprint 2	GJ-20, GJ-43, GJ-44, GJ-94	Pruebas y cobertura de 85 %, pruebas parte 2, colección Postman, puertos y conexión al broker.	**Entregado** con evidencia: 85 pruebas, JaCoCo 96 %, Postman de 13 pruebas.
Sprint 2	GJ-39, GJ-40, GJ-42	Cerrar sala al vender, publicar `MensajeEnviado`, idempotencia y cola de errores.	**Entregado** con evidencia en RabbitMQ y PostgreSQL.
Sprint 2	GJ-6	Pipeline CI/CD.	**Parcial.** Build, pruebas, Checkstyle e imagen Docker; el despliegue en Azure queda fuera por decisión del equipo.
Sprint 2	GJ-13	Definir alcance del chat de donaciones.	**Decidido:** entra en el alcance (activa GJ-57). El dominio ya reconoce `EN_DONACION` y `DONADA`.
Sprint 2	GJ-41, GJ-53	Feign hacia ms-segunda-vida; Gateway y docker-compose de integración.	**Pendiente.** El cliente Feign funciona con URL configurable; falta apuntarlo al servicio y definir ruta y puerto con el otro equipo.
Sprint 3	GJ-54, GJ-55	WebSockets STOMP y Redis Pub/Sub entre instancias.	**No implementado.** Ver ADR-001 y ADR-002.
Sprint 3	GJ-56, GJ-57	Chat con almacén aliado; chat de donaciones.	**No implementado.** GJ-57 se activa con la decisión de GJ-13.

02 ARQUITECTURA HEXAGONAL

Las dependencias apuntan hacia el dominio. Los casos de uso son los puertos de entrada; repositorios, publicador de eventos y cliente del servicio de segunda mano son los puertos de salida.

2.1 DIAGRAMA DE COMPONENTES



Las dependencias apuntan hacia adentro: **domain** no importa Spring, JPA, HTTP ni Lombok. El cableado se declara en **infrastructure/config/BeanConfig**.

Figura 1. Componentes de GJ-Chat-Service, generados a partir de los paquetes del código.

Explicación: los adaptadores de entrada traducen HTTP y AMQP a llamadas sobre interfaces de casos de uso. La aplicación coordina el flujo, el dominio decide y los adaptadores de salida resuelven persistencia, consulta de publicaciones y mensajería.

Justificación: aislar el dominio permite probar `Conversation` y `MessageText` sin contexto de Spring, y cambiar de dónde sale la publicación (hoy el monolito, después ms-segunda-vida) tocando solo `ListingQueryAdapter`.

2.2 CÓMO VIAJAN LOS DOS FLUJOS PRINCIPALES

```
ChatController → SendMessageUseCase «interface» → SendMessageUseCaseImpl
  → ConversationRepositoryPort  → ConversationRepositoryAdapter → PostgreSQL
  → Conversation                  reglas de dominio (participante, sala abierta)
  → ListingQueryPort            → ListingQueryAdapter → SecondHandFeignClient → segunda mano
  → MessageRepositoryPort       → MessageRepositoryAdapter → PostgreSQL
  → EventPublisherPort          → RabbitMqEventPublisherAdapter → styleradar.events

q.chat.segunda-mano → ListingSoldListener → CloseConversationsOnSaleUseCase «interface» → …Impl
  → ProcessedEventPort           isProcessed(eventId) → processed_events
  → ConversationRepositoryPort   findOpenByListing → close → save
  → ProcessedEventPort           markProcessed(eventId)
```

2.3 ESTRUCTURA DE PAQUETES

```
edu.dosw.styleradar.chat
├─ domain/          model · event · exception · port/in · port/out
├─ application/     usecase · dto/request · dto/response · mapper/in · mapper/out
└─ infrastructure/
   ├─ adapter/in/   rest · rest/doc · amqp
   ├─ adapter/out/  persistence · external · messaging · security
   ├─ config/       BeanConfig · SecurityConfig · RabbitConfig · FeignConfig · OpenApiConfig
   └─ exception/    GlobalExceptionHandler · SecurityProblemHandler · ProblemDetails
```

2.4 DEUDA ARQUITECTÓNICA CONOCIDA

HALLAZGO	CONSECUENCIA	ACCIÓN PROPUESTA
`JwtAuthenticationFilter`, `JwtTokenAdapter` y `AuthenticatedUser` están en `adapter/out/security` aunque sirven a la entrada HTTP.	El nombre del paquete confunde el sentido del adaptador.	Moverlos a `adapter/in/security`.
`ListingSoldListener` usa `EventEnvelope`, que vive en `adapter/out/messaging/dto`.	Un adaptador de entrada depende de un paquete de salida.	Mover el sobre a un paquete común de mensajería.
El `correlationId` del evento se genera en el publicador: ningún filtro lo copia de la petición HTTP.	El contrato pide copiarlo de la petición; hoy no se puede seguir una operación entre servicios.	Agregar un filtro que lea `X-Correlation-Id` y lo ponga en el `MDC`.
Los casos de uso no declaran transacción; cada `save` es una transacción distinta.	Cerrar la sala y guardar el mensaje, o publicar tras guardar, no son atómicos.	Evaluar `@Transactional` en el adaptador y un patrón outbox para el evento.
`PrendaSegundaManoRetiradaEvent` está propuesto pero no tiene cola ni listener.	La retirada de una publicación se detecta solo cuando alguien escribe (consulta Feign).	Acordar el evento con el equipo de segunda mano y consumirlo.

2.5 SECUENCIA · ENVIAR UN MENSAJE

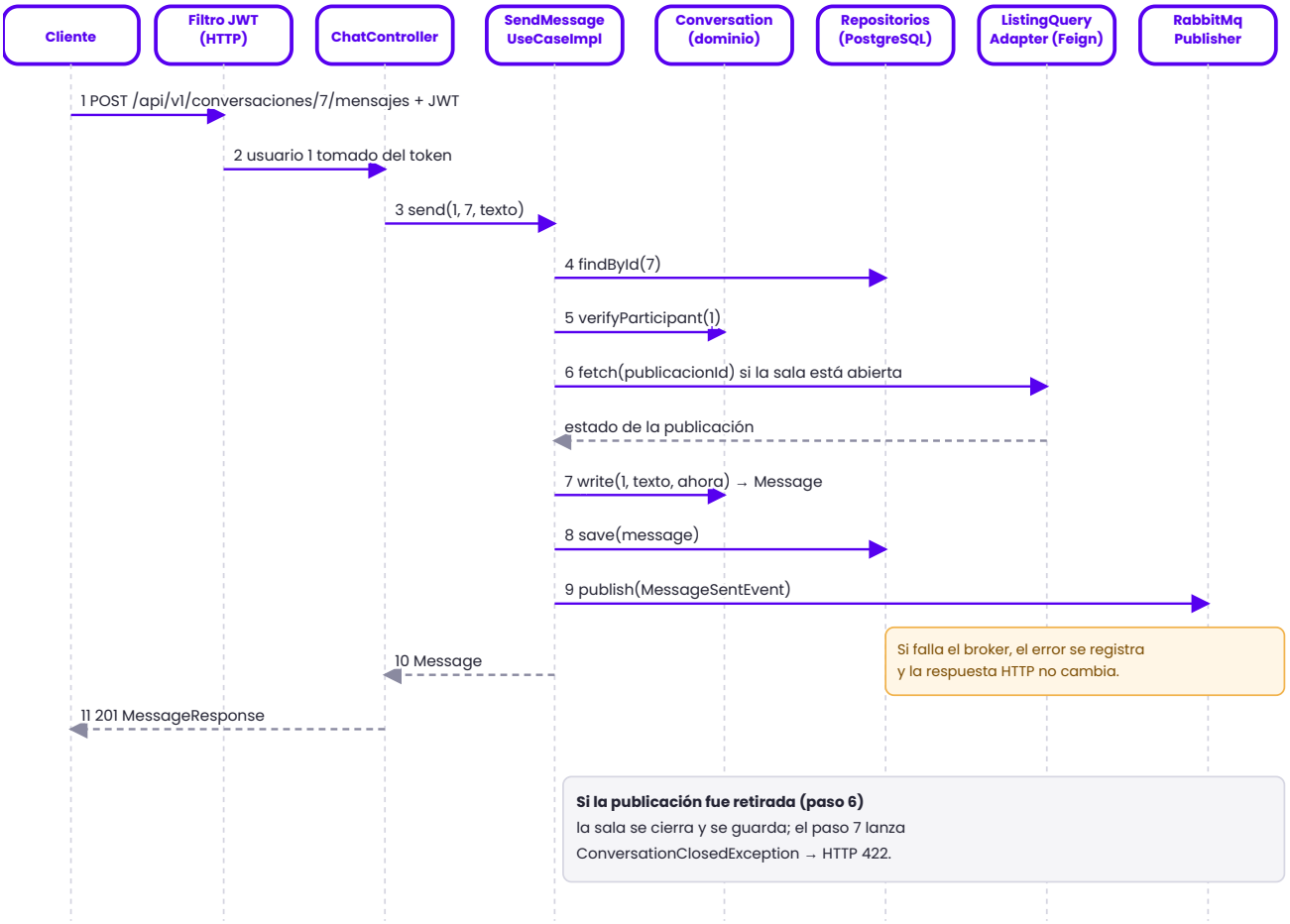


Figura 2. Secuencia de `POST /api/v1/conversaciones/{id}/mensajes`.

Explicación: la autenticación ocurre una sola vez, en el filtro; el controlador y el caso de uso reciben el id del usuario ya validado. El caso de uso consulta la publicación por Feign solo si la sala sigue abierta, deja que `Conversation.write` aplique las reglas y, después de guardar, publica `MessageSentEvent`.

Justificación: la consulta es síncrona porque el chat necesita saber en ese momento si la publicación fue retirada; el evento es asíncrono porque el chat no debe depender de que ms-notificaciones esté arriba para responder.

03 ENDPOINTS Y SWAGGER

Los endpoints de negocio se publican bajo `/api/v1`, requieren JWT Bearer y operan sobre el usuario del token. Hoy un token sin roles se interpreta con el rol `COMPRADOR`.

3.1 CONTRATO REST

MÉTODO	RUTA	DESCRIPCIÓN	CÓDIGOS
POST	`/segunda-mano/{listingId}/conversaciones`	Inicia la conversación del comprador con el dueño de la publicación; si ya existe, devuelve la misma.	200, 401, 403, 404, 422, 503
GET	`/usuarios/me/conversaciones`	Lista las conversaciones donde el usuario participa, como comprador o como vendedor.	200, 401, 403
GET	`/conversaciones/{id}/mensajes`	Mensajes en orden de envío; solo para los dos participantes.	200, 401, 403, 404
POST	`/conversaciones/{id}/mensajes`	Envía un mensaje de 1 a 1.000 caracteres y publica `chat.mensaje.enviado`.	201, 400, 401, 403, 404, 422, 503

Públicos: `/actuator/health`, `/actuator/info`, `/v3/api-docs` y `/swagger-ui.html` (que carga `/swagger-ui/index.html`). Todo lo demás exige JWT.

3.2 MANEJO DE ERRORES

`GlobalExceptionHandler` responde con `application/problem+json` (RFC 7807) con `type`, `title`, `status`, `detail`, `instance` y `timestamp`; los errores de validación agregan `errores` por campo. Los 401 y 403 los genera `SecurityProblemHandler`.

CÓDIGO	CAUSA
400	Texto vacío o de más de 1.000 caracteres, JSON mal formado o identificador con formato inválido.
401	Token ausente, inválido, de otro algoritmo o vencido.
403	El token trae un rol distinto de `COMPRADOR`.
404	La publicación no existe, o la conversación no existe o es ajena (no se revela cuál).
422	Regla de negocio: publicación no disponible (vendida, retirada, en donación), publicación propia o sala cerrada.
503	El servicio de segunda mano no respondió, devolvió un error o un estado desconocido.
500	Error no controlado; se registra y se responde sin la traza.

Nota sobre códigos: el código devuelve **422** tanto para publicación vendida o retirada como para publicación propia y sala cerrada. Si el squad prefiere **409** para «sala cerrada» o «publicación vendida», es un cambio de una línea en `GlobalExceptionHandler` que debe acordarse con quienes consumen la API.

3.3 EJEMPLO DE PETICIÓN Y RESPUESTA

```
POST /api/v1/segunda-mano/10/conversaciones
Authorization: Bearer <JWT>

200 OK
{ "id": 1, "publicacionId": 10, "prendaNombre": "Chaqueta de jean", "compradorId": 1,
  "vendedorId": 2, "estado": "ABIERTA", "fechaCreacion": "2026-11-07T21:39:41.004003589Z" }

POST /api/v1/conversaciones/1/mensajes
Authorization: Bearer <JWT>
{ "texto": "Hola, sigue disponible la chaqueta?" }

201 Created
{ "id": 2, "conversacionId": 1, "autorId": 1, "texto": "Hola, sigue disponible la chaqueta?",
  "fechaEnvio": "2026-11-07T22:04:49.188296896Z" }

POST /api/v1/segunda-mano/10/conversaciones        (sin token)

401 Unauthorized  · application/problem+json
{ "type": "https://styleradar.dosw/errores/no-autenticado", "title": "No autenticado", "status": 401,
  "detail": "JWT ausente, inválido o expirado", "instance": "/api/v1/segunda-mano/10/conversaciones",
  "timestamp": "2026-11-07T21:37:22.895849753Z" }
```

3.4 SEGURIDAD JWT

ASPECTO	IMPLEMENTACIÓN
Algoritmo y llave	HS256 con `JWT_SECRET_BASE64`. Sin valor por defecto: el servicio no arranca si la llave falta o tiene menos de 256 bits.
Validación	`JwtAuthenticationFilter` exige firma válida, algoritmo HS256, fechas coherentes y un `sub` numérico positivo.
Identidad	El id del usuario sale de `sub`; nunca del cuerpo ni de la URL.
Roles	Claim `roles`; si no viene, se asume `COMPRADOR` (`JWT_ROL_POR_DEFECTO`) porque el token del monolito no trae roles. Los endpoints exigen `hasRole('COMPRADOR')`.
Sesión y CORS	Sin sesión (`STATELESS`), CSRF deshabilitado; orígenes permitidos solo desde `CORS_ALLOWED_ORIGINS`.

3.5 EVIDENCIA DE SWAGGER

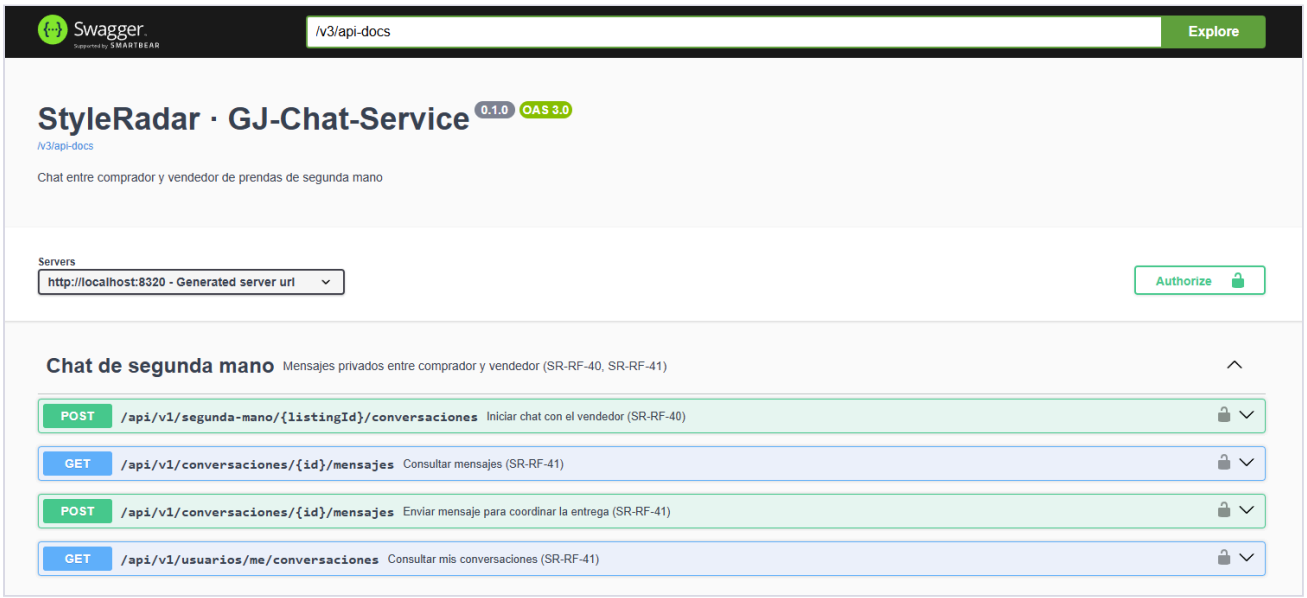


Figura 3. Swagger UI en ejecución local (`http://localhost:8320/swagger-ui.html`).

04 EVENTOS CONSUMIDOS Y PUBLICADOS

La comunicación asíncrona usa RabbitMQ 3.13 con el exchange `styleradar.events` (topic, durable). Los campos completos están en `docs/SR_GJ_ContratosEventos_v1.md`.

4.1 EVENTO CONSUMIDO

EVENTO · EMISOR	ROUTING KEY	COLA	ACCIÓN
PrendaSegundaManoVendidaEvent · servicio de segunda mano	`segunda-mano.prenda.vendida`	`q.chat.segunda-mano`	Cierra las salas abiertas de la publicación, salvo la del comprador que se quedó con la prenda.
PrendaSegundaManoRetiradaEvent *(propuesto)*	`segunda-mano.prenda.retirada`	—	**Sin cola ni listener.** Hoy la retirada se detecta al escribir, con la consulta Feign.

```
{ "eventId": "evt-venta-001", "eventType": "segunda-mano.prenda.vendida", "version": 1,
  "timestamp": "2026-11-07T22:20:00Z", "producer": "ms-segunda-mano", "correlationId": "corr-001",
  "data": { "publicacionId": 10, "vendedorId": 2, "compradorId": 1 } }
```

El listener ignora los campos que no usa (`@JsonIgnoreProperties`), por lo que `fechaVenta` y `prendaNombre` del contrato se aceptan sin romper la lectura.

4.2 EVENTO PUBLICADO

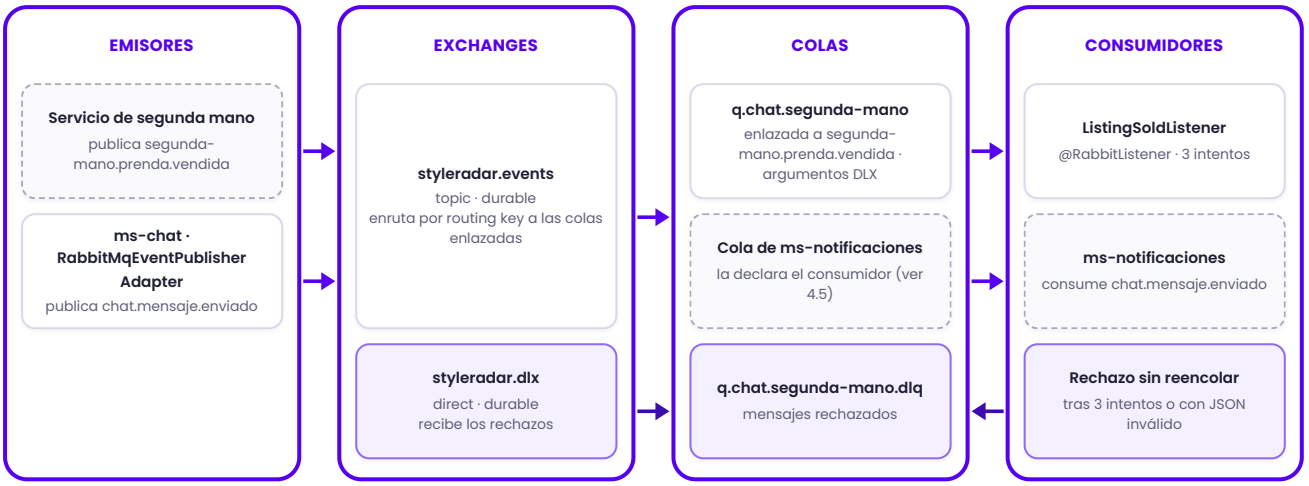
EVENTO · CONSUMIDOR	ROUTING KEY	CUÁNDO SE PUBLICA
MensajeEnviado · ms-notificaciones	`chat.mensaje.enviado`	Después de guardar un mensaje; si el broker falla, se registra el error y el envío HTTP sigue siendo exitoso.

```
{ "eventId": "ca0e13d6-2e26-43f0-8ef2-996432a7cc00", "eventType": "chat.mensaje.enviado", "version": 1,
  "timestamp": "2026-11-07T22:04:49.188296896Z", "producer": "ms-chat",
  "correlationId": "9659cdc3-fa5c-4876-9cb0-05d32bfe954f",
  "data": { "mensajeId": 2, "conversacionId": 1, "publicacionId": 10, "prendaNombre": "Chaqueta de jean",
            "autorId": 1, "destinatarioId": 2, "vistaPrevia": "Prueba del evento para GJ-40",
            "fechaEnvio": "2026-11-07T22:04:49.1078345557" } }
```

4.3 RESILIENCIA E IDEMPOTENCIA

- **Reintentos:** el listener hace 3 intentos en total, con espera creciente entre ellos (1 s y luego 2 s). Si la excepción persiste, el mensaje se rechaza sin reencolar.
- **Cola de errores:** el rechazo llega a `q.chat.segunda-mano.dlq` mediante `styleradar.dlx`. Un mensaje que no es JSON válido se rechaza de inmediato y queda con el encabezado `x-death` (`reason: rejected`).
- **Duplicados:** el caso de uso consulta `processed_events` antes de actuar y registra el `eventId` al terminar. Un evento repetido no cambia nada; además, cerrar una sala ya cerrada es inofensivo si el servicio cae entre el cierre y el registro.
- **Publicación:** el publicador captura `AmqpException`, la registra y no interrumpe la respuesta HTTP. No hay reintento ni outbox: un mensaje puede quedar sin evento si el broker está caído en ese instante.

4.4 TOPOLOGÍA DE MENSAJERÍA



Línea continua: implementado y verificado · borde punteado: lo declara otro servicio · tarjeta lila: ruta de errores.

Figura 4. Exchanges, colas y cola de errores de GJ-Chat-Service.

Explicación: ms-chat declara el exchange `styleradar.events`, el exchange de errores `styleradar.dlx`, la cola `q.chat.segunda-mano` (enlazada a `segunda-mano.prenda.vendida` y con `styleradar.dlx` como destino de rechazos) y la cola de errores. La cola del consumidor de `chat.mensaje.enviado` la declara ms-notificaciones.

Justificación: un exchange *topic* permite que varios servicios reciban el mismo evento sin que el productor los conozca; la cola de errores conserva el mensaje que no se pudo procesar en lugar de perderlo o de bloquear la cola.

4.5 COMPATIBILIDAD CON MS-NOTIFICACIONES

El documento técnico de ms-notificaciones declara convenciones distintas a las del contrato de eventos de ms-chat. Hasta que se acuerde una sola, los eventos de `MensajeEnviado` no llegan a esa cola.

ELEMENTO	MS-CHAT (CÓDIGO Y CONTRATO V1)	MS-NOTIFICACIONES (DOCUMENTO V1)
Exchange	`styleradar.events`	`styleradar.exchange`
Routing key del mensaje	`chat.mensaje.enviado`	`styleradar.chat.mensaje.enviado`
Cola del consumidor	`q.notificaciones.chat` (propuesta)	`styleradar.notificaciones.mensaje-enviado.q`
Exchange de errores	`styleradar.dlx` (direct)	`styleradar.dlx` (tipo no indicado)

Acción pendiente (GJ-12): acordar con ms-notificaciones el nombre del exchange y la routing key, y confirmar que `styleradar.dlx` se declara con el mismo tipo en ambos servicios: RabbitMQ rechaza redeclarar un exchange con otro tipo. Cambiar los nombres en ms-chat son tres constantes en `RabbitConfig` y sus pruebas.

05 MODELO DE DATOS

Servicio con persistencia propia en PostgreSQL 15 (`chat_db`). El esquema se versiona con Flyway en `V1__init_schema.sql` y `V2__processed_events.sql`, y Hibernate solo lo valida (`ddl-auto: validate`).

5.1 ESQUEMA FÍSICO



Sin llaves foráneas externas: **publicacion_id**, **comprador_id**, **vendedor_id** y **autor_id** son referencias lógicas a otros servicios (**Database-per-Service**).

Figura 5. Tablas e índices, generados a partir de las migraciones V1 y V2.

5.2 DECISIONES DEL MODELO

DECISIÓN	JUSTIFICACIÓN
Sin llaves foráneas externas	Publicación y usuarios viven en otros servicios y no se pueden referenciar entre bases de datos. La única llave foránea es interna: `mensajes → conversaciones`.
Una conversación por publicación y comprador	`uk_conversacion_publicacion_comprador` garantiza la unicidad aunque dos peticiones lleguen a la vez: la base rechaza la segunda y el adaptador devuelve la existente.
Índice de lectura	`ix_mensaje_conversacion_fecha` entrega los mensajes ordenados sin ordenar en memoria.
Idempotencia en base de datos	`processed_events.event_id` como llave primaria impide registrar dos veces el mismo evento.
Fechas con zona horaria	Todas las fechas son `TIMESTAMP WITH TIME ZONE` y se manejan en UTC.

5.3 VALORES DE DOMINIO

CAMPO	VALORES EN EL CÓDIGO (VALOR EXTERNO)
`ConversationStatus`	`OPEN` (`ABIERTA`), `CLOSED` (`CERRADA`)
`ListingStatus`	`AVAILABLE` (`DISPONIBLE`), `SOLD` (`VENDIDA`), `WITHDRAWN` (`RETIRADA`), `IN_DONATION` (`EN_DONACION`), `DONATED` (`DONADA`)
`MessageText`	Texto recortado de 1 a 1.000 caracteres; vacío o más largo lanza `InvalidMessageTextException`.

Planificado: el chat con almacenes aliados (GJ-56) y el de donaciones (GJ-57) requerirán identificar el tipo de conversación; hoy el esquema solo modela la conversación comprador–vendedor y se ampliará con una nueva migración.

06 DECISIONES TÉCNICAS

Cada decisión se evaluó con una matriz multicriterio ponderada y se registra como ADR. Las puntuaciones expresan la valoración cualitativa del equipo; no son mediciones de rendimiento.

6.1 CRITERIOS Y ESCALA

ID	CRITERIO	PESO	QUÉ EVALÚA EN ESTE SERVICIO
C1	**Consistencia y resiliencia**	25 %	Entrega ordenada, sin duplicar ni perder mensajes o eventos ante fallos.
C2	**Madurez y estabilidad**	20 %	Soporte, documentación y estabilidad de versiones.
C3	**Velocidad del equipo**	20 %	Curva de aprendizaje y productividad en el curso.
C4	**Operabilidad en Docker**	15 %	Contenedores, consumo local y observabilidad.
C5	**Costo y viabilidad académica**	20 %	Operar sin costo en desarrollo y CI.

Escala: **5** sobresaliente · **4** bueno · **3** aceptable · **2** deficiente · **1** inaceptable. Las decisiones de lenguaje, framework, base de datos y broker general (Java, Spring Boot, PostgreSQL, RabbitMQ) se heredan de `SR_GJ_AnalisisTecnologias_v1`.

6.2 ADR-001 · ENTREGA DE MENSAJES EN TIEMPO REAL (PLANIFICADO, GJ-54)

CRITERIO (PESO)	WEBSOCKET + STOMP ✔	SSE + REST	LONG POLLING	POLLING REST (HOY)
C1: Consistencia (25 %)	●●●●● (5)	●●●○○ (3)	●●●○○ (3)	●●○○○ (2)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●● (5)
C3: Velocidad (20 %)	●●●○○ (3)	●●●●○ (4)	●●●○○ (3)	●●●●● (5)
C4: Operabilidad (15 %)	●●●○○ (3)	●●●●○ (4)	●●●○○ (3)	●●●●○ (4)
C5: Costo (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)
Total ponderado	**4.30 / 5.00 (elegido para Sprint 3)**	**4.15 / 5.00**	**3.40 / 5.00**	**3.70 / 5.00**

Estado: Sprint 2: no se implementa. Los mensajes se envían por `POST` y se leen con `GET /conversaciones/{id}/mensajes`. WebSocket + STOMP queda descartado para este sprint y se construye en el Sprint 3 (GJ-54).

Justificación técnica del equipo: un chat es bidireccional y necesita baja latencia. STOMP da canales y confirmaciones sobre WebSocket con soporte nativo de Spring; SSE obliga a combinar dos canales y el polling REST añade latencia y carga.

6.3 ADR-002 · DIFUSIÓN ENTRE INSTANCIAS (PLANIFICADO, GJ-55)

CRITERIO (PESO)	REDIS PUB/SUB	RABBITMQ (FANOUT)	POSTGRESQL LISTEN/NOTIFY	INSTANCIA ÚNICA (HOY) ✔
C1: Consistencia (25 %)	●●●○○ (3)	●●●●○ (4)	●●●○○ (3)	●●○○○ (2)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●● (5)
C3: Velocidad (20 %)	●●●●○ (4)	●●○○○ (2)	●●●○○ (3)	●●●●● (5)
C4: Operabilidad (15 %)	●●●●○ (4)	●●●●● (5)	●●●●● (5)	●●●●● (5)
C5: Costo (20 %)	●●●●○ (4)	●●●●● (5)	●●●●● (5)	●●●●● (5)
Total ponderado	**3.95 / 5.00**	**4.15 / 5.00**	**3.90 / 5.00**	**4.25 / 5.00 (decisión de Sprint 2)**

Estado: Sprint 2: una sola instancia, sin difusión. Redis Pub/Sub queda descartado por ahora y no se agrega al compose. **Decisión abierta para GJ-55:** el plan asume Redis; con RabbitMQ casi empatado, debe confirmarse antes de empezar el Sprint 3.

Justificación técnica del equipo: con varias instancias, un mensaje recibido por una debe llegar a los clientes conectados a otra. Mientras haya una sola instancia y no exista WebSocket (ADR-001), no hay nada que difundir y agregar un servicio al compose solo suma costo. Para el Sprint 3, Redis Pub/Sub es liviano y los mensajes ya se guardan en PostgreSQL, así que perder un aviso no pierde el dato; RabbitMQ ya está en el stack y puntúa casi igual.

6.4 ADR-003 · CONSULTA DE LA PUBLICACIÓN (FEIGN HACIA MS-SEGUNDA-VIDA, GJ-41)

CRITERIO (PESO)	FEIGN (REST SÍNCRONO) ✔	CLIENTE HTTP MANUAL	RÉPLICA LOCAL POR EVENTOS	BASE COMPARTIDA
C1: Consistencia (25 %)	●●●●○ (4)	●●●●○ (4)	●●●●○ (4)	●○○○○ (1)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●● (5)
C3: Velocidad (20 %)	●●●●● (5)	●●●○○ (3)	●●○○○ (2)	●●●●● (5)
C4: Operabilidad (15 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)
C5: Costo (20 %)	●●●●● (5)	●●●●● (5)	●●●●● (5)	●●●●● (5)
Total ponderado	**4.75 / 5.00 (seleccionado)**	**4.35 / 5.00**	**3.80 / 5.00**	**3.85 / 5.00**

Estado: Aceptada. `SecondHandFeignClient` consulta `GET /api/v1/segunda-mano/{id}` con tiempos límite de 3 s y 5 s; un 404 se traduce a 404 y cualquier otro fallo a 503. La URL es `SEGUNDA_MANO_URL`. **Pendiente (GJ-41):** apuntarla a ms-segunda-vida cuando ese equipo fije puerto y ruta.

Justificación técnica del equipo: el chat necesita el estado y el dueño de la publicación en el momento de abrir o escribir. Feign lo resuelve con una interfaz declarativa, tiempos límite configurables y el mismo token. La base compartida rompe Database-per-Service y queda descartada por regla del curso; la réplica local exige eventos de publicación que aún no existen.

6.5 ADR-004 · RABBITMQ EN AZURE (DESCARTADO POR AHORA)

CRITERIO (PESO)	SOLO LOCAL (COMPOSE) ✔	CONTENEDOR EN AZURE	CLOUDAMQP GRATUITO	AZURE SERVICE BUS
C1: Consistencia (25 %)	●●○○○ (2)	●●●●○ (4)	●●●○○ (3)	●●●●○ (4)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)
C3: Velocidad (20 %)	●●●●● (5)	●●●○○ (3)	●●●●○ (4)	●●○○○ (2)
C4: Operabilidad (15 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●●○○○ (2)
C5: Costo (20 %)	●●●●● (5)	●●○○○ (2)	●●●●○ (4)	●●○○○ (2)
Total ponderado	**4.25 / 5.00 (seleccionado)**	**3.60 / 5.00**	**3.75 / 5.00**	**2.90 / 5.00**

Estado: Decisión abierta por el equipo. El despliegue en Azure queda fuera del alcance del Sprint 2: el pipeline construye, prueba y genera la imagen, y el despliegue solo corre en `master` con `DEPLOY_ENABLED`. Cuando se retome, esta ADR debe reevaluarse con los criterios C1 y C5.

Justificación técnica del equipo: mientras el equipo debate el despliegue en Azure, el broker corre en `docker-compose.yml` con healthcheck y el servicio espera a que esté sano. Esa opción puntúa más hoy por costo y velocidad, pero no sirve como entorno compartido: una instancia en la nube sigue necesitando un broker accesible. Service Bus exigiría cambiar el código de mensajería.

6.6 DECISIONES HEREDADAS

DECISIÓN	ELECCIÓN EN MS-CHAT	REFERENCIA
Lenguaje y framework	Java 17 · Spring Boot 3.2.12 (ms-notificaciones usa Java 21)	`SR_GJ_AnalisisTecnologias_v1`
Base de datos	PostgreSQL 15 con Flyway	`SR_GJ_AnalisisTecnologias_v1`
Broker	RabbitMQ 3.13 (topic + DLX)	`SR_GJ_ContratosEventos_v1`
Arquitectura	Hexagonal con interfaz + `Impl`	Votación del curso

07 RIESGOS Y MITIGACIÓN TÉCNICA

Riesgos identificados al cierre del Sprint 2. La columna de mitigación distingue lo implementado de lo pendiente.

7.1 EVALUACIÓN DE RIESGOS

RIESGO	PROB.	IMPACTO	MITIGACIÓN Y ESTADO
Convenciones de exchange y routing key distintas a las de ms-notificaciones.	Alta	Alto	**Pendiente:** acordar nombres (sección 4.5); hoy el aviso de mensaje nuevo no llega a esa cola.
Exchange `styleradar.dlx` declarado por varios servicios con tipos distintos.	Media	Alto	**Pendiente:** confirmar tipo *direct* con ms-notificaciones.
Evento perdido si el broker está caído al publicar.	Media	Medio	**Parcial:** se registra y el HTTP no se rompe; falta outbox o reintento.
Dependencia síncrona del servicio de segunda mano al iniciar o escribir.	Media	Medio	**Parcial:** tiempos límite de 3 s y 5 s y respuesta 503; falta circuit breaker.
Rol único: un token sin roles actúa como `COMPRADOR`, incluido el vendedor.	Media	Medio	**Pendiente:** definir con identidad los roles reales (`JWT_ROL_POR_DEFECTO`).
Retirada de publicación sin evento consumido.	Media	Bajo	**Parcial:** se detecta al escribir por la consulta Feign; falta `prenda.retirada`.
Secreto JWT ausente o débil.	Baja	Crítico	**Mitigado:** sin valor por defecto; el servicio no arranca con menos de 256 bits; `.env` fuera del repositorio.
Despliegue en Azure sin decisión.	Alta	Medio	**Aplazado:** fuera del alcance por decisión del equipo (ADR-004).

7.2 CONTROLES DE CALIDAD VIGENTES

PIPELINE EN GITHUB ACTIONS

`ci-qa.yml` ejecuta `mvn clean verify` con JaCoCo mínimo de 85 %, Checkstyle (Google Java Style) y construye la imagen Docker. El despliegue solo corre en `master` con `DEPLOY_ENABLED`.

HEALTHCHECK Y COMPOSE

PostgreSQL (`pg_isready`) y RabbitMQ (`rabbitmq-diagnostics ping`) deben estar sanos antes de iniciar el servicio. `/actuator/health` es público.

ERRORES RFC 7807

`ProblemDetail` estandariza las respuestas y evita exponer la traza al cliente.

08 CONCLUSIONES Y EQUIPO DE TRABAJO

8.1 STACK TECNOLÓGICO CON VERSIONES FIJAS

CAPA	TECNOLOGÍA Y VERSIÓN	ROL EN EL MICROSERVICIO
Lenguaje	**Java 17** (Eclipse Temurin)	`records` para DTO y eventos, tipado estricto y dominio puro.
Framework	**Spring Boot 3.2.12**	Web, Security, Data JPA, AMQP y Actuator; las versiones de Hibernate, Spring Security y Spring AMQP las fija su BOM.
Persistencia	**PostgreSQL 15** · Flyway	Datos propios, unicidad de conversación e idempotencia, y migraciones V1 y V2.
Mensajería	**RabbitMQ 3.13** · Spring AMQP	Eventos con exchange topic, cola durable y cola de errores.
Cliente HTTP	**OpenFeign** (Spring Cloud 2023.0.5)	Consulta de la publicación en el servicio de segunda mano.
Seguridad	**JJWT 0.12.6** (HS256)	Validación stateless del token.
Mapeo	**MapStruct 1.6.3** · Lombok 1.18.36	Conversión DTO ↔ dominio ↔ entidades; Lombok solo en entidades JPA.
Documentación	**springdoc-openapi 2.5.0**	Swagger en `/swagger-ui.html` y contrato en `/v3/api-docs`.
Calidad	**JUnit 5** · Mockito · Testcontainers · **JaCoCo 0.8.12** · Checkstyle 3.6.0	Cobertura mínima de 85 % en `mvn verify` y análisis estático.
Contenedores	**Docker** multi-stage · Compose	Imagen sin root, PostgreSQL 15 y RabbitMQ 3.13 en un solo comando.

8.2 VERIFICACIÓN REALIZADA

COMPROBACIÓN	RESULTADO	FECHA
`mvn clean verify`	85 pruebas sin fallos · JaCoCo 96 % de instrucciones (dominio y aplicación ≥ 97 %)	7 nov 2026
`docker compose up -d --build`	PostgreSQL, RabbitMQ y servicio sanos; `/actuator/health` en `UP`	7 nov 2026
Consola de RabbitMQ	Exchanges `styleradar.events` y `styleradar.dlx` y las dos colas declaradas	7 nov 2026
Seguridad JWT	Sin token → 401; rol `VENDEDOR` → 403; token válido → 200	7 nov 2026
Validación de la publicación	Inexistente → 404; vendida → 422; propia → 422	7 nov 2026
Evento `chat.mensaje.enviado`	Observado en una cola de prueba con el sobre completo	7 nov 2026
Cierre de sala al vender	La sala del otro comprador pasó a `CERRADA`; la del ganador siguió `ABIERTA`	7 nov 2026
Idempotencia y cola de errores	Evento repetido: una sola fila en `processed_events`; mensaje inválido en `q.chat.segunda-mano.dlq`	7 nov 2026
Colección Postman	7 peticiones, 13 pruebas, 0 fallidas	7 nov 2026
GitHub Actions · `CI QA`	**Pendiente** de ejecutar sobre `feature/sprint-2`	—

8.3 PENDIENTES PARA CERRAR EL SERVICIO EN SPRINT 2

- **Ejecución verde de `CI QA`** y captura de Swagger para GJ-17; pull request de `feature/sprint-2` a `develop`.
- **Convenciones de eventos** con ms-notificaciones (exchange, routing key, tipo de `styleradar.dlx`) y aprobación de los contratos (GJ-12).
- **GJ-41 y GJ-53:** apuntar el cliente Feign a ms-segunda-vida y sumar el servicio al gateway y al compose de integración, cuando los otros equipos fijen puerto y ruta.
- **Correlación:** copiar `X-Correlation-Id` de la petición al evento (sección 2.4).
- **Sprint 3:** WebSockets STOMP (GJ-54), difusión entre instancias (GJ-55), chat con almacén aliado (GJ-56) y chat de donaciones (GJ-57), aprobado en GJ-13.

8.4 RESPONSABLES DEL MICROSERVICIO

NOMBRE	ROL
Paula Diaz	Líder de Squad
Jose Martinez	Líder de Backend
Santiago Andrés García Almario	Desarrollo del microservicio de chat